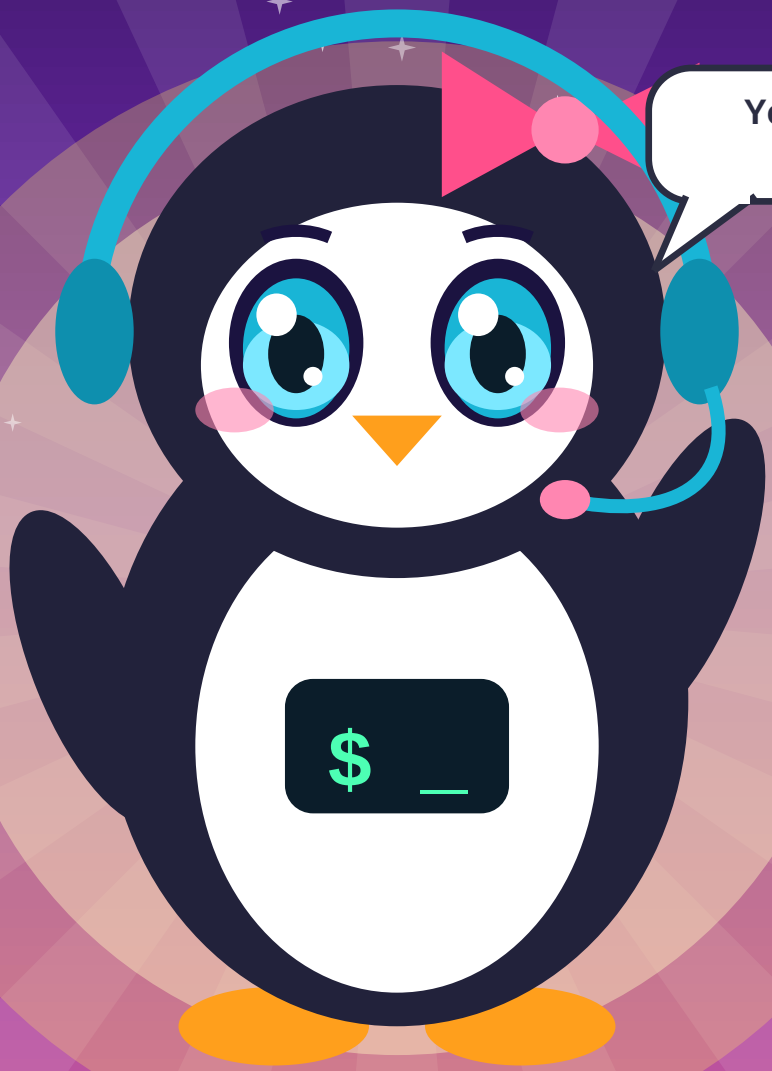
INTERVIEW
BOSS BATTLES

20 SCENARIO QUESTIONS | LINUX | DEVOPS | CLOUD | SYSADMIN



BOSS: THE INTERVIEWER HP 80%

- CREATED BY -
RUTUJA NAIK

20 SCENARIOS * 10 RAPID-FIRE * TRACE METHOD

00 HOW TO ANSWER

Use the TRACE method every time

T	**Triage**	How bad is it, who is affected, since when, and what changed recently?
R	**Read the evidence**	Check logs, metrics and state before touching anything.
A	**Analyze by layer**	Narrow it down: CPU, memory, disk, network, then application and dependencies.
C	**Change safely**	Smallest fix first, one change at a time, with a rollback ready.
E	**Educate and prevent**	Monitoring, automation, runbook and a blameless post-mortem.

YOUR 60-90 SECOND ANSWER TEMPLATE

1. Restate the problem and the impact (10 s). **2.** Say your order of hypotheses (10 s). **3.** Walk through 3 to 5 commands and what each tells you (40 s). **4.** State the fix and your rollback plan (15 s). **5.** Finish with prevention: monitoring, automation, documentation (10 s).

POWER PHRASES THAT SOUND SENIOR

"First I'd confirm the impact and check what changed recently."
"I'd gather evidence before making any change."
"I change one thing at a time so I can roll back cleanly."
"I'd fix the symptom now and the root cause after, then add monitoring."
"I don't know that one offhand, but here is how I would find out."

QUEST LOG (CONTENTS)

01	Production server is suddenly slow	2	11	A deployment broke production	12
02	Disk shows 100% but du disagrees	3	12	TLS certificate expired in production	13
03	Cannot SSH into a server	4	13	Grow a full volume without downtime	14
04	Service fails to start after reboot	5	14	Secure a freshly built server	15
05	kill -9 will not kill the process	6	15	Server may be compromised	16
06	Application keeps getting OOM-killed	7	16	Intermittent packet loss	17
07	Permission denied, but perms look right	8	17	Cron job did not run	18
08	Nginx returns 502 / 504 errors	9	18	Terraform drift after a manual change	19
09	Pod stuck in CrashLoopBackOff	10	19	Zero-downtime deployment	20
10	Pod stuck in Pending	11	20	High load average but idle CPU	21

Troubleshooting | **Admin and security** | **DevOps and cloud** | then: rapid-fire round and final checklist

01 PRODUCTION SERVER IS SUDDENLY SLOW

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

A production server that was fine this morning is now very slow and users are complaining. What do you do?

SAY IT LIKE THIS (60-90 seconds, out loud)

First I'd confirm the impact: who is affected, since when, and whether anything changed, like a deployment or a traffic spike. Then I'd check the four resources: CPU, memory, disk and network. I start with `uptime` to read the load averages and compare them with the CPU count from `nproc`. Then `top` to find hot processes, `free -m` for memory and swap, `iostat -xz 1` for disk latency, and `dmesg -T` for kernel errors such as OOM kills. Once I know which resource is under pressure, I find the process responsible and decide whether to restart, throttle, roll back or scale. Finally I add alerting so we catch it earlier next time.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`uptime`	Load averages for 1, 5 and 15 minutes. If the 1-minute value is higher than the 15-minute value, things are getting worse. Compare with `nproc`.
2	`top (or htop)`	Press `P` to sort by CPU and `M` by memory. Watch `%wa` (waiting on disk) and `%st` (CPU stolen by the hypervisor on VMs).
3	`free -m`	Look at the **available** column, not free. Low available memory plus growing swap means memory pressure.
4	`vmstat 1 5`	`si/so` high means swapping. `wa` high means IO wait. `r` larger than CPU count means a CPU queue.
5	`iostat -xz 1`	`%util` near 100 and a high `await` mean the disk is the bottleneck.
6	`ss -s and sar -n DEV 1`	Connection counts and network throughput. Spot saturation or a connection flood.
7	`dmesg -T | tail`	Kernel messages: OOM kills, disk errors, NIC resets.

COMMON ROOT CAUSES

Runaway or looping process, memory leak leading to swap, slow or shared disk, traffic spike, a bad deploy, a backup or cron job running at peak time, slow database queries, noisy neighbour on a shared VM.

FIX AND PREVENTION

Restart or fix the offending process, roll back a bad deploy, scale up or out, and limit noisy jobs with `nice`, `ionice` or cgroups. Prevent it with dashboards and alerts (Prometheus or CloudWatch) on CPU, memory, IO wait and latency.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. What does a load average of 8 mean on a 4-core server?

A. Roughly twice as many tasks want CPU as there are cores, so work is queueing. Then check whether it is CPU-bound or IO wait.

Q. What is steal time?

A. CPU time the hypervisor gave to other VMs. High `%st` means a noisy neighbour or an undersized instance.

AVOID THIS MISTAKE

Rebooting straight away. You lose the evidence and the problem usually comes back.

02 DISK SHOWS 100% BUT DU DISAGREES

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

df -h says the disk is 100% full, but when you add up the folders with du, it does not add up. Why, and what do you do?

SAY IT LIKE THIS (60-90 seconds, out loud)

That pattern usually means a deleted file is still held open by a process. When you delete a file that a process still has open, the name disappears so `du` can't see it, but the blocks stay allocated until the process closes it, so `df` still shows full. I'd run `lsof +L1` to list open files with a link count of zero, find the process and restart or reload it. If that's not the cause, I check inodes with `df -i`, because millions of tiny files can exhaust inodes while space looks fine. Then I look at the usual suspects: logs, the systemd journal, Docker data and old backups.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`df -h`	Which filesystem is full, and where is it mounted?
2	`df -i`	Inode usage. 100% IUse% means no new files can be created even if space is free.
3	`lsof +L1`	Open files that were deleted. The PID and name tell you which process to restart.
4	`du -xh / --max-depth=1 | sort -rh | head`	Biggest top-level folders. `-x` stays on one filesystem so other mounts don't confuse the totals.
5	`journalctl --disk-usage`	Is the systemd journal huge? Shrink with `journalctl --vacuum-size=200M`.
6	`docker system df`	How much space images, containers and volumes use.
7	`find / -xdev -size +500M`	Individual large files on this filesystem.

COMMON ROOT CAUSES

Log file deleted without restarting the writer, inode exhaustion, huge journal or log files, Docker layers and volumes, and files hidden under a mount point (written before the volume was mounted).

FIX AND PREVENTION

Restart the process holding the deleted file. For live logs, empty them with `truncate -s 0 file` instead of `rm`. Set up `logrotate` with a reload hook, vacuum the journal, prune Docker, and alert at 80% disk usage.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. How can you free the space without restarting the process?

A. Truncate through /proc: `: > /proc/PID/fd/N`. Use carefully and only if you understand the impact.

Q. What is an inode?

A. A metadata record for a file (owner, permissions, data block pointers). Each file uses one, and ext4 has a fixed number set at format time.

AVOID THIS MISTAKE

Running rm -rf on random folders under pressure. Identify the cause first.

03 CANNOT SSH INTO A SERVER

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

You cannot SSH into a Linux server in the cloud. Walk me through how you troubleshoot.

SAY IT LIKE THIS (60-90 seconds, out loud)

I work from the network upward. First I test the port directly with `nc -zv host 22` or `ssh -vvv`, because ping can be blocked. How it fails tells me the layer: a timeout means packets are dropped, so I check security groups, NACLs, route tables and firewalls. 'Connection refused' means the host is up but nothing is listening on 22. 'Permission denied' means the network is fine and the problem is authentication: keys, file permissions or sshd config. If I have console access or SSM Session Manager, I log in that way and check `systemctl status sshd`, `sshd -t` and the auth log.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`ssh -vvv user@host`	Verbose output shows exactly where it stops: connect, key exchange or authentication.
2	`nc -zv host 22`	Is the TCP port reachable? Timeout versus refused is the key clue.
3	**Cloud side: security group, NACL, route table, public IP**	Is port 22 allowed from your current IP? Did your IP change? Does the instance pass its status checks?
4	`systemctl status sshd and sshd -t`	(On the server via console/SSM.) Is sshd running and is the config valid?
5	`sudo tail /var/log/auth.log`	Debian/Ubuntu. On RHEL use `/var/log/secure`. Shows why logins were rejected.
6	`ls -ld ~ ~/.ssh ~/.ssh/authorized_keys`	Home must not be group-writable, `.ssh` should be 700, `authorized_keys` 600.
7	`fail2ban-client status sshd and df -h`	Is your IP banned? Is the disk full (this can break logins)?

COMMON ROOT CAUSES

Security group or NACL change, your public IP changed, sshd stopped or on another port, wrong key or bad permissions, fail2ban ban, full disk, instance failing status checks.

FIX AND PREVENTION

Fix the rule, restart sshd, correct permissions, unban the IP. Keep a break-glass path such as SSM Session Manager or the serial console, so one SSH issue is never a lockout.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Timeout versus connection refused?

A. Timeout: packets are dropped by a firewall or routing. Refused: the host answered, but nothing listens on that port.

Q. How do you avoid locking yourself out when editing sshd_config?

A. Run `sshd -t` first, keep a second session open, and reload instead of restart.

AVOID THIS MISTAKE

Changing sshd settings blindly before you know which layer is failing.

04 SERVICE FAILS TO START AFTER REBOOT

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

An application service works when you start it manually but fails to start after a server reboot. How do you find out why?

SAY IT LIKE THIS (60-90 seconds, out loud)

When it works manually but not at boot, I suspect ordering, environment or enablement. I'd start with `systemctl status` and `journalctl -u app -b` to read the actual error from this boot. Then I check the unit file with `systemctl cat`: is it enabled, is `ExecStart` correct, which user does it run as, and does it need the network or a mount that isn't ready yet? Boot-time failures are often ordering problems, so I'd add `After=network-online.target` or a mount dependency. I'd also check for a port conflict and SELinux denials. After fixing, I run `daemon-reload`, then reboot to prove it.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`systemctl status app -l`	Current state, last exit code and the last log lines.
2	`journalctl -u app -b --no-pager`	Full log for this boot only.
3	`systemctl cat app`	The effective unit file, including overrides.
4	`systemd-analyze verify /etc/systemd/system/app.service`	Catches syntax errors and missing files in the unit.
5	`systemctl is-enabled app`	Enabled means it starts at boot. `start` only runs it now.
6	`systemctl list-dependencies app`	Shows what it is ordered after, such as network or mounts.
7	`ss -tulpn | grep :PORT`	Is another process already using the port?
8	`findmnt and cat /etc/fstab`	Is a required mount missing? Use `nofail` for non-critical mounts.

COMMON ROOT CAUSES

Service not enabled, starts before network or storage is ready, wrong path or user in the unit, missing environment file, port already in use, SELinux denial, dependency such as a database not ready.

FIX AND PREVENTION

Correct the unit, run `systemctl daemon-reload`, then `systemctl enable --now app`. Add `After=` and `Wants=network-online.target`, set `Restart=on-failure`, and test with a real reboot.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. After= versus Requires=?

A. `After=` only controls ordering. `Requires=` creates a hard dependency. They are often used together.

Q. What does daemon-reload do?

A. It makes systemd re-read unit files. It is needed after you edit or add one.

AVOID THIS MISTAKE

Mixing up start and enable. Start runs it now, enable makes it run at boot.

05 KILL -9 WILL NOT KILL THE PROCESS

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

You run kill -9 on a process and it is still there. Why might that happen?

SAY IT LIKE THIS (60-90 seconds, out loud)

Three common reasons. First, the process is in uninterruptible sleep, state `D`, usually stuck waiting on disk or a hung NFS mount. Signals are only delivered when the kernel call returns, so even SIGKILL waits. Second, it is a zombie, state `Z`. It is already dead; only its process table entry remains because the parent hasn't called `wait()`. You can't kill a zombie, you fix or restart the parent. Third, something is restarting it, like systemd with `Restart=always` or a Docker restart policy, so it looks unkillable. I'd check the STAT column with `ps` first and then act on the real cause.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`ps -eo pid,ppid,stat,wchan:20,cmd | grep -E ' [DZ]'`	`D` = uninterruptible IO wait. `Z` = zombie. `wchan` shows what the kernel is waiting on.
2	`cat /proc/PID/stack`	(root) The kernel stack of the stuck process.
3	`dmesg -T | tail`	Hung task warnings, disk errors, NFS timeouts.
4	`mount | grep nfs and nfsstat`	Is a network filesystem unreachable?
5	`umount -f -l /mnt/share`	Forced lazy unmount, the usual way to release a hung NFS mount.
6	`ps -o pid,ppid,cmd -p PPID`	For a zombie, identify the parent. Restart or fix the parent.
7	`systemctl status app or docker ps`	Is a supervisor respawning it?

COMMON ROOT CAUSES

Hung NFS or network storage, failing disk or driver, buggy parent that never reaps children, process supervisors restarting the service.

FIX AND PREVENTION

Restore the storage or lazy-unmount it. Restart the parent of a zombie. Stop the supervisor if you really want it gone. Reboot only as a last resort.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. How do you prevent zombie processes?

A. The parent must reap children with `wait()`. In containers use a proper init process such as `--init` or tini.

Q. SIGTERM versus SIGKILL?

A. TERM (15) lets the process clean up and can be handled. KILL (9) is enforced by the kernel and cannot be caught.

AVOID THIS MISTAKE

Spamming kill -9. Read the STAT column first.

06 APPLICATION KEEPS GETTING OOM-KILLED

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

An application keeps dying and restarting. You suspect it is running out of memory. How do you confirm and fix it?

SAY IT LIKE THIS (60-90 seconds, out loud)

I'd confirm with the kernel log: `dmesg -T | grep -i oom` or `journalctl -k`. The OOM killer logs which process it killed and why. Then I look at memory with `free -m` and `ps aux --sort=-%mem`. In containers I remember the limit is enforced by cgroups, so a container can be OOM-killed even when the host has free RAM. In Kubernetes, `kubectl describe pod` shows `OOMKilled` with exit code 137. The next step is to decide whether it's a leak, where memory grows forever, or simply a limit that's too low for the real workload. I'd graph memory over time to tell the difference.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`dmesg -T | grep -i -E 'killed process|oom'`	The victim process, its memory use and the time it was killed.
2	`free -m and ps aux --sort=-%mem | head`	System memory and the biggest consumers. Read **available**, not free.
3	`grep -E 'VmRSS|VmSwap' /proc/PID/status`	Real memory a process is using, and how much is swapped.
4	`kubectl describe pod p`	Look for `Last State: OOMKilled`, exit code 137, and the memory limit.
5	`kubectl top pod and docker stats`	Live memory use compared with the limit.
6	`docker inspect -f '{{.State.OOMKilled}}' c`	True if the container hit its memory limit.
7	`cat /sys/fs/cgroup/memory.max`	The cgroup v2 memory limit that applies to this process.

COMMON ROOT CAUSES

Memory leak, limit set too low, traffic spike, large caches, JVM heap larger than the container limit, no swap, too many workers.

FIX AND PREVENTION

Fix the leak or tune the heap (for Java keep `-Xmx` below the limit). Raise requests and limits based on measured usage, add replicas, and alert on working-set memory. Swap can buffer spikes but is not a fix.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Requests versus limits in Kubernetes?

A. Requests are what the scheduler reserves. Limits are the hard cap: exceeding the memory limit gets you OOMKilled, exceeding the CPU limit only throttles.

Q. Why is 'free' memory low but the system is healthy?

A. Linux uses spare RAM for page cache, which is reclaimable. Check the **available** column.

AVOID THIS MISTAKE

Only raising the limit. Check whether memory grows without bound, which means a leak.

07 PERMISSION DENIED, BUT PERMS LOOK RIGHT

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

A user gets Permission denied accessing a file, but ls -l shows the permissions should allow it. What do you check?

SAY IT LIKE THIS (60-90 seconds, out loud)

I'd stop looking at just the file and check the whole path, because every directory needs the execute bit. `namei -l` shows the permissions of each component in one view. Then I confirm who the user really is and which groups are active with `id`, since a newly added group needs a new login. After that I check the less obvious layers: ACLs with `getfacl`, SELinux with `ls -Z` and `ausearch`, immutable flags with `lsattr`, and mount options like `noexec` or read-only. If it's a script that says 'permission denied' or 'bad interpreter', I check the shebang and Windows line endings too.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`namei -l /full/path/to/file`	Permissions and owners of every directory along the path. A missing `x` on a parent folder is common.
2	`id username`	Real UID and active groups. New group membership needs a fresh login.
3	`getfacl file`	ACL entries can override the normal permission bits.
4	`getenforce and ls -Z file`	SELinux mode and file context. Denials hide behind normal permissions.
5	`sudo ausearch -m avc -ts recent`	Recent SELinux denials with the exact reason.
6	`lsattr file`	An immutable (`i`) flag blocks changes even for root.
7	`mount | grep -E 'noexec|ro,'`	Read-only or noexec mounts block writes or execution.

COMMON ROOT CAUSES

Missing execute permission on a parent directory, wrong group, ACL override, SELinux context, immutable attribute, read-only or noexec mount, group change not yet active, CRLF line endings in a script.

FIX AND PREVENTION

Set the minimum permissions needed with `chmod` and `chown`. For SELinux use `restorecon -Rv path` or the right boolean, not disabling SELinux. Remove an immutable flag with `chattr -i` if appropriate.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. What does x mean on a directory?

A. Permission to enter and access items inside it. `r` only lets you list the names.

Q. What is the sticky bit?

A. On a shared directory like /tmp, it means only a file's owner can delete or rename it (`chmod +t`).

AVOID THIS MISTAKE

Using chmod 777 as a quick fix. It hides the cause and creates a security hole.

08 NGINX RETURNS 502 / 504 ERRORS

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

Users report 502 Bad Gateway and 504 Gateway Timeout errors from your Nginx reverse proxy. What is your approach?

SAY IT LIKE THIS (60-90 seconds, out loud)

First I make sure I understand what those codes mean. Both are generated by the proxy to say the backend has a problem. 502 means it couldn't get a valid response, like the app is down or refused the connection. 504 means the backend didn't answer in time. So I read the Nginx error log for lines like 'connect() failed' or 'upstream timed out'. Then I go to the backend: is it running, is it listening on the expected port, can I reach it with `curl` from the proxy host, and is it overloaded? I'd also check worker exhaustion, file descriptor limits and, on RHEL, SELinux. I avoid just raising timeouts, as that hides a slow dependency.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`tail -f /var/log/nginx/error.log`	Look for `connect() failed (111)` (refused) or `upstream timed out` (slow).
2	`systemctl status app or docker ps`	Is the backend running or crash-looping?
3	`ss -tulpn | grep :PORT`	Is anything listening on the upstream port or socket?
4	`curl -v http://127.0.0.1:PORT/health`	From the proxy host, does the backend answer, and how fast?
5	`nginx -T | grep -E 'proxy_pass|timeout'`	The real effective upstream address and timeout values.
6	`top and free -m on the backend`	Is the backend out of CPU or memory, or are its workers all busy?
7	`getsebool httpd_can_network_connect`	RHEL: Nginx may be blocked by SELinux from connecting to the backend.

COMMON ROOT CAUSES

Backend crashed or restarting, wrong port or socket, overloaded backend or slow database query, app workers exhausted, timeouts too low, file descriptor limits, SELinux, stale DNS for the upstream.

FIX AND PREVENTION

Restore or scale the backend and fix the slow query. Tune `proxy_read_timeout` only after fixing the slowness. Use health checks in the load balancer and alert on 5xx rate.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Where does a 502 come from?

A. From the proxy or load balancer, reporting that the backend failed. So debug the backend.

Q. 502 versus 503?

A. 503 means service unavailable: overloaded, in maintenance, or no healthy backends.

AVOID THIS MISTAKE

Raising timeouts first. It hides the slow dependency instead of fixing it.

09 POD STUCK IN CRASHLOOPBACKOFF

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

A Kubernetes pod keeps restarting and shows CrashLoopBackOff. How do you debug it?

SAY IT LIKE THIS (60-90 seconds, out loud)

CrashLoopBackOff means the container starts, exits, and Kubernetes keeps restarting it with increasing delay. I begin with `kubectl describe pod` and read the Events, the exit code and Last State. Then `kubectl logs --previous`, because the current container may have no logs yet. The exit code helps: 1 is an application error, 126 or 127 is a command problem, 137 is killed, often OOM, and 143 is SIGTERM, for example from a failed liveness probe. Then I verify the config: are the ConfigMaps and Secrets present, are the env vars right, are the volumes mounted, and can the app reach its dependencies?

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`kubectl describe pod p`	Events, Last State, Exit Code, probe settings and restart count.
2	`kubectl logs p --previous`	Logs from the crashed container. This is the key command.
3	`kubectl get events --sort-by=.lastTimestamp`	Cluster events in time order.
4	`kubectl get pod p -o yaml`	Check command, args, env, volumes and probes.
5	`kubectl get cm,secret -n ns`	Do the referenced ConfigMaps and Secrets exist?
6	`kubectl run tmp --rm -it --image=busybox -- sh`	Throwaway pod to test DNS and connectivity to dependencies.
7	`kubectl debug -it p --image=busybox --target=c`	Attach a debug container next to the failing one.

COMMON ROOT CAUSES

Application error from bad config or missing environment variable, wrong command or entrypoint, failing liveness probe, OOMKilled, missing Secret or volume, database unreachable, running as non-root without permissions.

FIX AND PREVENTION

Fix the config or image. For slow-starting apps add a `startupProbe` and loosen the liveness probe. Raise memory if OOMKilled, and use init containers to wait for dependencies.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Liveness versus readiness probe?

A. Liveness restarts a stuck container. Readiness removes it from Service endpoints until it is ready. Startup protects slow starters.

Q. What is the backoff?

A. The kubelet waits longer after each failure: 10s, 20s, 40s and so on up to 5 minutes.

AVOID THIS MISTAKE

Reading only the current logs. Use --previous to see why it crashed.

10 POD STUCK IN PENDING

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

A pod stays in Pending and never starts. What could be wrong and how do you find out?

SAY IT LIKE THIS (60-90 seconds, out loud)

Pending means the scheduler hasn't placed the pod on a node yet, so the answer is almost always in the Events of `kubectl describe pod`. It will show a FailedScheduling message such as '0/5 nodes available: insufficient memory, 2 had untolerated taints'. Scheduling is based on resource requests, not actual usage, so I compare requests with node allocatable using `kubectl describe node`. I also check taints and tolerations, nodeSelector and affinity rules, PersistentVolumeClaims that are not bound, and resource quotas. If the cluster is simply full, the fix is more nodes or the cluster autoscaler.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`kubectl describe pod p`	The Events section gives the exact FailedScheduling reason.
2	`kubectl get nodes and kubectl top nodes`	Node health and current usage.
3	`kubectl describe node n`	Compare Allocatable with Allocated requests, and look at the Taints.
4	`kubectl get pvc -n ns`	A PVC in Pending state keeps its pod Pending too.
5	`kubectl get resourcequota -n ns`	Has the namespace hit its quota?
6	`kubectl get pod p -o yaml | grep -A5 -E 'nodeSelector|affinity|tolerations'`	Rules that restrict which nodes are allowed.
7	`kubectl get storageclass`	Does the requested StorageClass exist, and in the right zone?

COMMON ROOT CAUSES

Not enough CPU or memory requested, taints without matching tolerations, selector or affinity mismatch, unbound PVC or wrong availability zone, quota exceeded, cluster autoscaler at its maximum.

FIX AND PREVENTION

Lower requests or add nodes, enable the autoscaler, add tolerations, fix the StorageClass or zone, and raise the quota if justified.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Pending versus ContainerCreating?

A. Pending: not scheduled to a node yet. ContainerCreating: scheduled, now pulling the image or mounting volumes.

Q. What is a taint?

A. A node marker that repels pods unless they have a matching toleration, for example for dedicated GPU nodes.

AVOID THIS MISTAKE

Assuming the cluster is out of real capacity. Scheduling uses requests, not usage.

11 A DEPLOYMENT BROKE PRODUCTION

DEVOPS & CLOUD DIFFICULTY ✦✦✦

THE QUESTION

You just deployed a new version and production is throwing errors. What do you do?

SAY IT LIKE THIS (60-90 seconds, out loud)

My first priority is to restore service, not to find the root cause. So I announce the incident, then roll back to the last known good version: `kubectl rollout undo`, `helm rollback`, redeploying the previous image tag, or turning off a feature flag. I confirm recovery on dashboards: error rate, latency and health checks. Only then do I investigate: compare the diff between versions, read logs and metrics around the deploy time, and check config, secrets and database migrations. Afterwards I run a blameless post-mortem with a timeline, root cause and action items, such as better tests, canary releases and automatic rollback.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`kubectl rollout history deploy/web`	See revisions available to roll back to.
2	`kubectl rollout undo deploy/web`	Roll back to the previous revision.
3	`helm history r then helm rollback r N`	Same idea for Helm releases.
4	`kubectl rollout status deploy/web`	Confirm the rollback finished and pods are healthy.
5	`kubectl logs deploy/web --since=30m`	Errors since around the deploy time.
6	`git diff v1.4..v1.5 and kubectl diff -f app.yaml`	What exactly changed in code and manifests?
7	Dashboards: error rate, latency, saturation	Confirm users are recovered and see when the problem started.

COMMON ROOT CAUSES

A code bug, a config or secret change, incompatible database migration, missing environment variable, resource limits too low, dependency version change.

FIX AND PREVENTION

Prevent repeats with automated tests, staging environments, canary or blue/green rollouts, readiness gates, feature flags, SLO-based alerts and a documented rollback runbook.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Roll back or roll forward?

A. Roll back when the cause isn't obvious or the fix is slow. Roll forward only for a tiny, well-understood fix. Database migrations can make rollback impossible.

Q. What goes into a blameless post-mortem?

A. Timeline, impact, root cause, what went well and badly, and action items with owners and dates.

AVOID THIS MISTAKE

Debugging live while users are down. Restore service first.

12 TLS CERTIFICATE EXPIRED IN PRODUCTION

ADMIN & SECURITY DIFFICULTY ✦ ✦ ✦

THE QUESTION

Customers see a certificate expired warning on your production website. How do you fix it, and how do you stop it happening again?

SAY IT LIKE THIS (60-90 seconds, out loud)

First I confirm what the server is actually presenting using `openssl s_client` and read the notAfter date, because it might be the origin server, a load balancer or a CDN serving the old certificate. Then I renew it. With Let's Encrypt that's `certbot renew`, and with a cloud load balancer, I'd check ACM or the key vault. The classic trap is renewing the file but not reloading the web server, so it keeps serving the old cert from memory. After the fix I verify from outside with `curl` and a browser. For prevention: automatic renewal with a timer, monitoring on certificate expiry with alerts at 30, 14 and 7 days, and clear ownership.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`echo | openssl s_client -connect host:443 -servername host 2>/dev/null | openssl x509 -noout -dates -subject`	What certificate is really being served, and when does it expire?
2	`sudo certbot certificates`	Certificates certbot manages, with expiry dates.
3	`sudo certbot renew --dry-run`	Tests whether automatic renewal will work. Shows why it failed.
4	`systemctl status certbot.timer`	Is the renewal timer enabled and running?
5	`sudo nginx -t && sudo systemctl reload nginx`	Validate and reload so the new certificate is loaded.
6	`curl -vI https://host`	Verify from outside: chain, dates and HTTP response.
7	Load balancer or CDN	Check which certificate is attached there, since it may be separate from the server's.

COMMON ROOT CAUSES

Auto-renew failed because port 80 was blocked for the HTTP-01 challenge, DNS changed, the timer was disabled, or rate limits. A manually managed cert was forgotten. The service wasn't reloaded after renewal.

FIX AND PREVENTION

Renew, reload, verify. Then automate: certbot timer or cloud-managed certificates, expiry monitoring (for example blackbox exporter), alerts and a runbook with an owner.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Why reload after renewing?

A. Servers read certificates at start or reload. The file changed on disk, but the running process still serves the old one.

Q. HTTP-01 versus DNS-01 challenge?

A. HTTP-01 proves control over port 80. DNS-01 uses a TXT record and supports wildcard certificates.

AVOID THIS MISTAKE

Renewing the certificate but not reloading the service.

13 GROW A FULL VOLUME WITHOUT DOWNTIME

ADMIN & SECURITY DIFFICULTY ✦✦✦

THE QUESTION

A data volume is almost full on a production server and you cannot take downtime. How do you grow it?

SAY IT LIKE THIS (60-90 seconds, out loud)

I'd start by taking a snapshot or confirming a recent backup, since resizing is normally safe but not risk free. In the cloud I increase the disk first, for example with `aws ec2 modify-volume`. Then on the server I make the OS see the new size: `growpart` if there's a partition, then `pvresize` if LVM is used. With LVM I extend the logical volume, using `lvextend -r` so the filesystem grows together with it. Without LVM I grow the filesystem directly. ext4 uses `resize2fs` and XFS uses `xfs_growfs`, and both work online. I confirm with `df -h` and then add monitoring so we don't hit 100% again. Note that XFS can't be shrunk.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`lsblk and df -hT`	Current layout, filesystem type and size.
2	`aws ec2 modify-volume --volume-id vol-123 --size 200`	Cloud step: grow the disk (Azure and GCP have equivalents).
3	`sudo growpart /dev/nvme0n1 1`	Grow the partition to use the new disk space.
4	`sudo pvresize /dev/nvme0n1p1`	LVM: let the physical volume see the new size.
5	`sudo lvextend -r -L +50G /dev/vg/lv`	Extend the logical volume and resize the filesystem together.
6	`sudo resize2fs /dev/vg/lv (ext4)`	Online grow for ext4 if you didn't use `-r`.
7	`sudo xfs_growfs /mount/point (XFS)`	Online grow for XFS. It takes the mount point.
8	`df -h`	Verify the new size.

KEY POINTS TO MENTION

Order matters: disk, partition, physical volume, logical volume, filesystem. ext4 and XFS support online growth. XFS cannot shrink, so plan sizes or migrate data. In the cloud a volume can usually only be modified every few hours.

FIX AND PREVENTION

Always snapshot first, test the procedure in staging, and set disk alerts at 80% with growth trends. Where possible automate with Terraform or Ansible.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Can you shrink an XFS filesystem?

A. No. ext4 can shrink offline. For XFS you create a smaller volume and copy the data.

Q. What if the disk has no LVM?

A. Grow the partition with `growpart`, then grow the filesystem. It still works online for ext4 and XFS.

AVOID THIS MISTAKE

Skipping the snapshot, or resizing the filesystem before the underlying volume.

14 SECURE A FRESHLY BUILT SERVER

ADMIN & SECURITY DIFFICULTY ✦✦✦

THE QUESTION

You have just provisioned a new Linux server. How do you harden it before it goes into production?

SAY IT LIKE THIS (60-90 seconds, out loud)

I follow defence in depth and reduce the attack surface. First, update all packages and enable automatic security updates. Second, create named admin users with **sudo**, and set SSH to keys only with root login disabled. I validate with **sshd -t** and keep a second session open while I change it. Third, a default-deny firewall that only allows the ports we need, plus cloud security groups. I disable unused services, keep SELinux or AppArmor enforcing, add fail2ban, enable auditd and ship logs centrally, and sync time with chrony. Finally I run a baseline audit such as lynis or a CIS benchmark. Ideally all of this is automated with cloud-init or Ansible, so every server is identical.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`sudo apt update && sudo apt upgrade -y`	Patch known vulnerabilities. Enable **unattended-upgrades** for ongoing security updates.
2	`sudo adduser admin && sudo usermod -aG sudo admin`	Named accounts give accountability. Avoid shared root.
3	`ssh-copy-id admin@host`	Install your public key before disabling passwords.
4	`PermitRootLogin no / PasswordAuthentication no`	In **/etc/ssh/sshd_config**. Then **sudo sshd -t && sudo systemctl reload sshd**.
5	`sudo ufw default deny incoming && sudo ufw allow 22/tcp && sudo ufw enable`	Default-deny firewall that allows only what you need.
6	`ss -tulpn and systemctl list-unit-files --state=enabled`	Find and disable services you don't need.
7	`getenforce or aa-status`	Confirm SELinux or AppArmor is enforcing.
8	`sudo lynis audit system`	Baseline security audit with suggestions.

KEY POINTS TO MENTION

Least privilege, minimal attack surface, patching, logging and auditing, and automation. Combine host controls (firewall, SELinux) with network controls (security groups, private subnets, bastion or SSM).

FIX AND PREVENTION

Bake the baseline into a golden image or configuration management so it is repeatable and drift is detected. Back up, monitor and review access regularly.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Why disable root SSH login?

A. Root is a known username that attackers brute-force. Named users with sudo give an audit trail and limit blast radius.

Q. Where do you automate this?

A. In cloud-init, Packer images or Ansible roles, so every server starts from the same hardened baseline.

AVOID THIS MISTAKE

Locking yourself out. Test a second SSH session before closing the first.

15 SERVER MAY BE COMPROMISED

ADMIN & SECURITY DIFFICULTY ✦✦✦

THE QUESTION

You suspect a production server has been hacked. What are your first steps?

SAY IT LIKE THIS (60-90 seconds, out loud)

I follow contain, investigate, eradicate, recover and learn. First I contain it without destroying evidence: isolate the network, for example by swapping the security group to a quarantine one, instead of powering it off, because a reboot destroys volatile data. I snapshot the disk for forensics. Then I investigate: unexpected logins with `last`, odd processes and connections, new cron jobs, new SSH keys, new UID 0 users, recently changed files and package integrity. I assume tools on the host could be tampered with. Then I eradicate by rebuilding from a trusted image rather than cleaning in place, rotate every credential that was on the box, patch the entry point, and write a post-incident report.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`last -a and lastb and w`	Successful and failed logins, with source IPs. Who is on right now?
2	`ss -tupn and ps auxf`	Unusual outbound connections and suspicious process trees (miners, reverse shells).
3	`for u in $(cut -d: -f1 /etc/passwd); do crontab -l -u $u 2>/dev/null; done`	Scheduled persistence for every user.
4	`awk -F: '$3==0' /etc/passwd`	Any account other than root with UID 0 is a red flag.
5	`find / -xdev -type f -mtime -2 2>/dev/null | head`	Files changed in the last two days.
6	`ls -la /tmp /dev/shm and find / -perm -4000 -type f`	Dropped binaries and unexpected SUID files.
7	`debsums -c or rpm -Va`	Package integrity check: modified system binaries.
8	**Cloud audit logs (CloudTrail, Activity Log)**	Check API activity and new access keys, roles or instances.

KEY POINTS TO MENTION

Preserve evidence, don't trust the compromised host's tools (rootkits), and follow your incident response plan. Determine scope: which data and credentials were exposed.

FIX AND PREVENTION

Rebuild from a trusted image, restore data from a clean backup, rotate all secrets and keys, patch the vulnerability, and add detection (auditd, EDR, alerts). Notify stakeholders as policy requires.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Why not just reboot it?

A. A reboot wipes memory and live connections (evidence) and may trigger persistence mechanisms.

Q. Why rebuild rather than clean?

A. You can't be sure everything malicious was found. A rebuild from a known-good image is the only trustworthy state.

AVOID THIS MISTAKE

Cleaning the server in place and moving on.

16 INTERMITTENT PACKET LOSS

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

Two servers experience intermittent packet loss and slow transfers between them. How do you investigate?

SAY IT LIKE THIS (60-90 seconds, out loud)

I start by measuring: `ping -c 100` for loss and jitter, and `mtr` to see on which hop the loss starts. Loss that continues to the final hop is real. Loss at just one middle hop is often a router rate-limiting ICMP and can be ignored. Then I look at each host: `ip -s link` for errors and drops, and `ethtool` for speed and duplex. I test MTU problems with `ping -M do -s 1472`, because small packets working while large ones hang is the classic symptom. I'd also check conntrack table exhaustion, cloud instance bandwidth limits and security group or NACL issues, and capture packets with `tcpdump` if needed.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`ping -c 100 host`	Packet loss percentage and jitter over a decent sample.
2	`mtr -rwc 100 host`	Per-hop loss. Real loss persists through all later hops.
3	`ip -s link show eth0`	RX and TX errors, dropped and overrun counters.
4	`ethtool eth0 and ethtool -S eth0 | grep -iE 'drop|err'`	Link speed, duplex and NIC-level drop counters.
5	`ping -M do -s 1472 host`	Path MTU test. Fails if packets need fragmenting.
6	`dmesg | grep -i conntrack`	'table full, dropping packet' means the conntrack table is exhausted.
7	`ss -ti and sar -n TCP,ETCP 1`	TCP retransmissions show where the network is hurting.
8	`sudo tcpdump -nn -i eth0 host X`	Capture traffic to see drops, resets and retransmits.

COMMON ROOT CAUSES

Congested link, bad cable or NIC, duplex mismatch, MTU mismatch, conntrack table full, cloud instance network limit or burst exhausted, noisy neighbour, security device rate limiting, bad route.

FIX AND PREVENTION

Fix or replace the faulty link, correct MTU or duplex, increase conntrack size (`net.netfilter.nf_conntrack_max`), pick a bigger instance, and monitor retransmits and drops.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Why does loss at one mtr hop not always matter?
A. Routers often deprioritise ICMP replies. Only loss that continues to later hops and the destination is real.

Q. What is a typical MTU problem symptom?
A. Small requests work but large transfers or TLS handshakes stall, often across VPNs or tunnels.

AVOID THIS MISTAKE

Trusting a single mtr hop. Look at loss all the way to the destination.

17 CRON JOB DID NOT RUN

TROUBLESHOOTING DIFFICULTY ✦ ✦ ✦

THE QUESTION

A scheduled cron job did not run last night. How do you find out why?

SAY IT LIKE THIS (60-90 seconds, out loud)

I check in order. Is the cron daemon running, and is the job in the right user's crontab? Then the cron log: `grep CRON /var/log/syslog`, or `/var/log/cron` on RHEL, shows whether the job was triggered at all. If it was triggered but nothing happened, the problem is usually the environment: cron has a minimal `PATH` and no login profile, so commands that work in my shell fail. I run the script with a stripped environment to reproduce it. I also check script permissions and shebang, the crontab syntax (a `%` must be escaped), the timezone, and I redirect output to a log. For important jobs I'd use `flock` to avoid overlap and consider a systemd timer.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`crontab -l and sudo crontab -l -u user`	Is the job under the correct user?
2	`systemctl status cron (or crond on RHEL)`	Is the cron service running?
3	`grep CRON /var/log/syslog | tail`	Debian/Ubuntu. RHEL: `/var/log/cron`. Was the job started?
4	`env -i /bin/sh -c '/path/job.sh'`	Run with a minimal environment like cron does.
5	`ls -l /path/job.sh and head -1 /path/job.sh`	Executable bit and correct shebang.
6	`*/5 * * * * /path/job.sh >> /var/log/job.log 2>&1`	Always log output, otherwise errors vanish.
7	`timedatectl`	Server timezone may differ from what you assumed.

COMMON ROOT CAUSES

Different `PATH` or environment in cron, relative paths, script not executable, `%` not escaped in the crontab, wrong timezone, cron daemon stopped, job overlapped or hung, no output logging.

FIX AND PREVENTION

Use full paths and set `PATH=` at the top of the crontab, log everything, protect against overlap with `flock -n /tmp/job.lock job.sh`, and alert when a job fails or doesn't run.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Cron versus systemd timers?

A. Timers add journal logging, dependencies, `Persistent=true` to catch up after downtime, and randomised delays.

Q. How do you prevent overlapping runs?

A. `flock -n /tmp/job.lock /path/job.sh` exits immediately if the previous run still holds the lock.

AVOID THIS MISTAKE

Assuming cron has your shell's PATH and environment.

18 TERRAFORM DRIFT AFTER A MANUAL CHANGE

DEVOPS & CLOUD DIFFICULTY ✦✦✦

THE QUESTION

Someone changed a cloud resource manually in the console, and now Terraform shows differences. How do you handle it?

SAY IT LIKE THIS (60-90 seconds, out loud)

That's configuration drift. I run `terraform plan`, or `plan -refresh-only` to see only the drift. Then I decide what the source of truth should be. If the manual change was wrong, I apply the code to revert it. If the change was needed, I update the Terraform code to match and re-run plan until there's no diff. If someone created a resource by hand, I adopt it with `terraform import`. I also find out who changed it and why using CloudTrail or the activity log. To prevent it, I limit console access with least-privilege IAM, use a remote state backend with locking, run scheduled plans in CI to detect drift, and use `lifecycle ignore_changes` for values legitimately managed elsewhere.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`terraform plan`	Shows what differs between code, state and reality.
2	`terraform plan -refresh-only`	Shows only drift in real resources, without proposing code changes.
3	`terraform state list and terraform state show addr`	What Terraform thinks it manages, and its recorded attributes.
4	`terraform import aws_instance.web i-123456`	Bring an existing resource under Terraform control.
5	`terraform apply -replace=aws_instance.web`	Force recreation of one resource (replaces the old taint command).
6	CloudTrail / Activity Log	Who changed it, when and how, so you can fix the process, not just the symptom.
7	CI: scheduled terraform plan	Detects drift automatically and alerts the team.

KEY POINTS TO MENTION

State maps your code to real resources. Remote backends (for example S3 with locking) give sharing, locking, versioning and encryption. Never edit the state file by hand, use `terraform state` commands. Use `force-unlock` only when sure no run is active.

FIX AND PREVENTION

Restrict manual changes, require changes through pull requests, review every plan (look for destroy and replace), and add policy as code (OPA or Sentinel) for guardrails.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. What is Terraform state and why remote?

A. State records which real resources belong to your code. Remote state allows team sharing, locking against concurrent applies, and backups.

Q. How do you deal with values managed outside Terraform?

A. Use `lifecycle { ignore_changes = [...] }`, for example for autoscaling desired capacity.

AVOID THIS MISTAKE

Blindly running apply. Read the plan for unexpected destroys.

19 ZERO-DOWNTIME DEPLOYMENT

DEVOPS & CLOUD DIFFICULTY ✦✦✦

THE QUESTION

How would you deploy a new application version with zero downtime?

SAY IT LIKE THIS (60-90 seconds, out loud)

I combine a safe rollout strategy with health checks and graceful shutdown. In Kubernetes I use a rolling update with `maxUnavailable: 0` so old pods stay until new ones are ready, a readiness probe so traffic only reaches healthy pods, and a PodDisruptionBudget. Pods need graceful shutdown: handle SIGTERM, use a `preStop` hook and a sufficient termination grace period, so in-flight requests finish. For higher risk changes I use blue/green for instant switch and rollback, or canary, sending 5% of traffic first and increasing as metrics stay healthy. The hardest part is the database: I use backward-compatible expand and contract migrations because old and new versions run together briefly.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`kubectl set image deploy/web web=app:v2`	Triggers a rolling update to the new image.
2	`kubectl rollout status deploy/web`	Waits until the new pods are ready. Gate your pipeline on it.
3	`kubectl rollout undo deploy/web`	Instant rollback if metrics degrade.
4	strategy: RollingUpdate, maxUnavailable 0, maxSurge 1	Never drop below the desired capacity during an update.
5	readinessProbe + preStop + terminationGracePeriodSeconds	Send traffic only when ready, and drain connections on shutdown.
6	`kubectl get pdb`	PodDisruptionBudget keeps enough pods alive during node maintenance.
7	Spread replicas across zones	`topologySpreadConstraints` or anti-affinity survive a zone failure.

KEY POINTS TO MENTION

Multiple replicas, health checks, graceful shutdown, observability during rollout, and fast rollback. For databases: expand (add column), deploy, backfill, then contract (remove the old column) in a later release.

FIX AND PREVENTION

Automate with CI/CD, run smoke tests after each step, and use progressive delivery tools (Argo Rollouts, Flagger or weighted load balancer targets) for automatic canary analysis and rollback.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Rolling versus blue/green trade-off?

A. Rolling is cheaper but runs mixed versions. Blue/green needs double capacity but switches and rolls back instantly.

Q. Why are database migrations the hard part?

A. Old and new code run at the same time, so schema changes must work with both versions.

AVOID THIS MISTAKE

Forgetting graceful shutdown. Pods killed mid-request cause errors even with rolling updates.

20 HIGH LOAD AVERAGE BUT IDLE CPU

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

The load average is very high, but the CPU looks mostly idle. How is that possible, and how do you investigate?

SAY IT LIKE THIS (60-90 seconds, out loud)

On Linux, load average counts not only tasks that are running or waiting for CPU, but also tasks in uninterruptible sleep, state D, which are usually blocked on IO. So high load with idle CPU almost always means processes are waiting on disk or network storage. I'd confirm with `top`, where `%wa` is high, and `vmstat 1`, where the `b` column shows blocked processes. Then `iostat -xz 1` shows disk latency and utilization, and `ps` can list the D-state processes and what they wait on. I'd also check for a hung NFS mount, an EBS volume that has exhausted its IOPS or burst credits, and swapping. Pressure stall information in `/proc/pressure/io` gives a direct measure.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`top`	High `%wa` means CPUs are idle because they are waiting for IO.
2	`vmstat 1`	`b` = processes blocked on IO, `wa` = IO wait, `si/so` = swapping.
3	`iostat -xz 1`	`await` (latency), `%util` and queue size show disk saturation.
4	`ps -eo state,pid,wchan:20,cmd | awk '$1=="D"'`	The D-state processes and what they are waiting on.
5	`iotop -oP and pidstat -d 1`	Which processes generate the disk IO.
6	`cat /proc/pressure/io`	PSI: percentage of time tasks stalled waiting for IO.
7	`dmesg -T | grep -iE 'error|timeout|nfs' and nfsstat`	Disk errors and NFS problems.
8	CloudWatch: BurstBalance, VolumeQueueLength	Cloud volumes can hit IOPS or throughput limits.

COMMON ROOT CAUSES

Slow or failing disk, EBS volume out of IOPS or burst credits, hung NFS, heavy swapping, large backup or log writes, many threads blocked on locks.

FIX AND PREVENTION

Reduce or reschedule the IO load, move to a faster or provisioned-IOPS volume, fix the storage or NFS, add memory to stop swapping, and alert on IO wait and latency.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Does Linux load differ from other Unix systems?
A. Yes. Linux includes uninterruptible (D state) tasks in the load, many others count only CPU-runnable tasks.

Q. What is %wa?
A. The share of time CPUs were idle while at least one task waited for IO.

AVOID THIS MISTAKE

Assuming a high load always means a CPU problem.

21 RAPID-FIRE ROUND

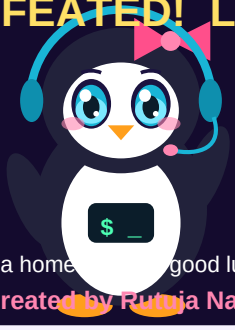
10 quick questions, 1 to 2 sentence answers

1	**Process versus thread?**	A process has its own memory space. Threads share their process's memory and are cheaper to create and switch.
2	**What is swap, and when is it used?**	Disk space used as overflow when RAM is short. Heavy swapping slows everything, so treat it as a safety net, not capacity.
3	**What does chmod 4755 do?**	The leading 4 sets the setuid bit: the file runs with its owner's privileges (like passwd). 755 means rwxr-xr-x.
4	**What is /proc?**	A virtual filesystem exposing kernel and process information, such as /proc/meminfo. Nothing is stored on disk.
5	**How does hostname resolution work?**	Order is set in /etc/nsswitch.conf: usually /etc/hosts first, then the DNS servers from /etc/resolv.conf or systemd-resolved.
6	**Namespaces versus cgroups?**	Namespaces isolate what a process can see (PIDs, network, mounts). Cgroups limit what it can use (CPU, memory, IO). Containers combine both.
7	**What does 2>&1 mean?**	Send stderr (fd 2) to wherever stdout (fd 1) points, so `cmd > out.txt 2>&1` captures both.
8	**Which process uses port 8080?**	`ss -tulpn | grep :8080` or `lsof -i :8080`.
9	**sudo versus su?**	sudo runs one command with elevated rights and logs it per user. su switches to another user's shell and needs that user's password.
10	**What is an init system?**	PID 1. It starts and supervises all other processes and services. On most modern distributions it is systemd.

BEFORE THE INTERVIEW: FINAL CHECKLIST

- Practise each answer out loud in 60 to 90 seconds. Record yourself once.
- Prepare a real example for each topic: "Last time I saw this, I..."
- Know why a command works, not just its syntax.
- Admit what you don't know, then explain how you would find out.
- End answers with prevention: monitoring, automation, documentation.
- Keep a home lab (VM, Docker, minikube). Break things on purpose and fix them.

BOSS DEFEATED! LEVEL UP!



Practice out loud, build a home[illegible] good luck with your interviews.

Created by Rutuja Naik